CRDTs and Eventually Consistent Replication

Advanced Database Management — Computer Science (A & B), 4th Semester

Project 02 Manual / *Languages: C or C++*

1 Problem Statement

You will build a replicated key-value store in which multiple servers accept writes concurrently, never coordinate before responding to clients, and still converge on exactly the same state over time. This is a very different kind of replication from the master-slave and Raft projects in this problem set. In those projects, a single leader decides the order of all writes, and followers follow. Here, there is no leader. Every server is equal. A client can write to any server at any time, and that write is accepted and acknowledged immediately, no quorum, no election, no wait. The cluster figures out the global order of events after the fact, using a mathematical structure called a *conflict-free replicated data type*, or CRDT.

The idea is easiest to understand with a concrete example. Imagine a shared counter showing the number of likes on a post. Two users in different regions click the like button at the same moment. Each click is handled by a different nearby server, and each server increments its local copy of the counter from 10 to 11. Now the two servers need to reconcile. A naive approach ("take whichever number is higher") is wrong: both are 11, so the naive merge loses one of the increments and the final count is 11, not 12. A CRDT-based approach is different: each server independently tracks how many increments it has contributed, and the merge takes the maximum per server and sums the results. Server A knows it has contributed 1 increment; server B knows it has contributed 1 increment. The merge produces "A has 1, B has 1," which sums to 2, plus the original 10 gives 12. The increments are not lost.

CRDTs come in several flavors for different kinds of data. A counter that only goes up. A counter that goes up and down. A register that holds a single value (with last-writer-wins semantics). A set that supports both add and remove. This project implements four of them, along with a gossip protocol that lets servers in a cluster exchange their states periodically and merge whatever they receive. The result is a key-value store where writes happen at wire speed (no coordination) and replicas converge to an identical state within a few gossip cycles, even in the face of network partitions and server crashes. The tradeoff, and this is the central lesson of the project, is that you give up *linearizability*, the illusion that all operations happen in a single global order. Reads on different servers may briefly see different states, and there is no way to ask "what is the value right now" and get a globally consistent answer. The technical term for this weaker guarantee is *eventual consistency*, and it is the foundation of systems like Amazon DynamoDB, Apache Cassandra, Riak, and modern collaborative editing tools like Figma and Google Docs.

You are not implementing a general-purpose CRDT framework. You are implementing four specific CRDTs, a gossip protocol, and a small key-value store on top. Vector clocks, Byzantine fault tolerance, strong eventual consistency proofs beyond the informal arguments in this manual, and operation-based CRDTs are all out of scope; you are building *state-based* CRDTs only.

2 What the Final Product Looks Like

When your project is complete, you can start three server processes (on three machines, three Docker containers, or three terminals on one laptop) and observe them accepting concurrent writes and converging via background gossip. A session looks like this.

```
# terminal 1: start node A
$ ./crdtkv --id A --port 6001 --gossip-port 7001 \
           --peers B@h2:7002,C@h3:7003
[A] starting, gossip interval 1000ms
[A] peers: B@h2:7002, C@h3:7003
[A] ready

# terminal 2: start node B
$ ./crdtkv --id B --port 6002 --gossip-port 7002 \
           --peers A@h1:7001,C@h3:7003
```

```
[B] ready

# terminal 3: start node C
$ ./crdtkv --id C --port 6003 --gossip-port 7003 \
           --peers A@h1:7001,B@h2:7002
[C] ready

# terminal 4: client connects to node A
$ ./crdtkv-cli h1 6001
connected to node A

> CREATE likes GCOUNTER
OK

> INC likes 1
OK (likes = 1)

> INC likes 1
OK (likes = 2)

> GET likes
2

# terminal 5: client connects to node B (different node!)
$ ./crdtkv-cli h2 6002
connected to node B

> INC likes 1
OK (likes = 1)
# ^ node B sees its local value only.
#   It has not yet received gossip from A.

> GET likes
1

# wait a couple of seconds for gossip...
> GET likes
3
# ^ now node B has merged A's state into its own.
#   Total is 2 (from A) + 1 (from B) = 3.

# meanwhile on node A:
> GET likes
3
# ^ A also converged to 3 after receiving B's state via gossip.
```

The magic here is that nothing coordinates. Both clients got immediate acknowledgements. Both clients did their own increments independently. And yet both nodes agreed on the final value without anyone telling them what order to apply things in. This is the promise of CRDTs: local writes are always fast, and convergence is automatic.

2.1 Partition Tolerance Demonstration

Now the harder scenario. Disconnect node C from the cluster (simulate a network partition) and write to both sides.

```
# on node A:
> INC likes 5
OK (likes = 8)
> INC likes 2
OK (likes = 10)

# on node C (disconnected from A and B):
> INC likes 7
OK (likes = 10)
```

```
# ^ C's view: the last state it heard about was 3, plus its own 7 = 10.

# heal the partition. Within one gossip cycle:
# on node A:
> GET likes
17
# ^ 10 from the A/B side + 7 from C = 17.

# on node C:
> GET likes
17
# ^ C also sees 17 after merge.
```

This is the defining property of CRDTs. Both sides of the partition kept accepting writes. The writes were not lost, were not overwritten, were not duplicated. The merge arithmetic produced the correct final count. A system built on leader election (like Raft) would have stopped accepting writes on the minority side during the partition; a CRDT system does not. The tradeoff — again — is that during the partition, the two sides temporarily disagreed about the value, and any client that read during the partition got a "stale" answer. Whether that tradeoff is acceptable depends on the application.

3 Deployment Options

The server is agnostic to deployment. Use whichever of these works for you.

3.1 Three Processes on One Machine

Start three copies of the server in three terminals on the same computer, using different ports and `localhost` in the peer list. This is the recommended development setup.

3.2 Docker Compose

A `docker-compose.yml` with three services makes partition testing particularly clean, because `docker network disconnect` / `docker network connect` give you a real way to partition nodes (rather than simulating it in code).

3.3 Three EC2 Instances or Three Raspberry Pis

Any three machines that can reach each other on TCP work fine. Real cross-machine deployment is the most realistic environment for observing gossip convergence across genuinely unreliable links.

3.4 Cluster Size Is Fixed at Three

The cluster always has exactly three nodes. CRDTs scale to arbitrary cluster sizes, but three is enough to demonstrate every interesting behavior (convergence, partition tolerance, gossip propagation) without the complexity of membership management.

4 CRDTs: The Math You Need

This section is the conceptual core of the project. Read it carefully before writing any code. A single misunderstanding here will waste days of debugging later.

4.1 State-Based CRDTs

A state-based CRDT (also called a *convergent replicated data type*, or CvRDT) is a data type together with three operations:

- A **query**: a pure function of the current state that returns the user-visible value (e.g., "what number does this counter currently show?").
- An **update**: a function that mutates the local state in response to a user action (e.g., "increment by 1").
- A **merge**: a function that takes two states and produces a new combined state.

For the type to be a CRDT, its **merge** function must have three properties, all of them essential:

1. **Commutative**: $\texttt{merge}(a, b) = \texttt{merge}(b, a)$. The order of merging does not matter.
2. **Associative**: $\texttt{merge}(\texttt{merge}(a, b), c) = \texttt{merge}(a, \texttt{merge}(b, c))$. Grouping of merges does not matter.
3. **Idempotent**: $\texttt{merge}(a, a) = a$. Merging a state with itself changes nothing.

If these three properties hold for every possible pair of states, then regardless of the order or number of times gossip delivers messages between servers, all servers eventually converge to the same state. The proof is actually quite short and I recommend you read it in Shapiro's paper (see Section 15); in brief, the three properties combined mean that the set of all possible states forms a mathematical structure called a *join-semilattice*, which guarantees that any sequence of merges produces the same final element.

You will not prove these properties formally for your implementations, but you will write unit tests that check them on concrete examples. Every CRDT you implement must pass commutativity, associativity, and idempotence tests on randomly generated state pairs. This is non-negotiable: a bug in any of the three properties means your cluster will not actually converge, and the gossip protocol will produce subtly wrong answers that are very hard to debug after the fact.

4.2 The Four CRDTs

You will implement exactly four CRDTs. They are the standard textbook examples, and they cover a wide enough range of use cases that real applications can be built on them.

4.2.1 G-Counter (Grow-Only Counter)

A counter that only supports increment, never decrement.

The state is a map from *node id* to *integer*. Each server tracks how much *it, specifically* has contributed to the counter.

- **Query**: sum of all values in the map.
- **Update (increment by n, on node `id`)**: `state[id] += n`.
- **Merge**: for each `id` present in either state, the merged state's `state[id]` is the *maximum* of the two inputs' values for that `id`.

Why does this work? The key insight is that a node's own entry is monotonic — node A's contribution only ever goes up, and no other node ever modifies A's entry. So when two states are merged, taking the max per entry is correct: for each node, we keep whichever state knew about more of that node's increments. Sums, of course, are then correct because no contribution is double-counted.

Commutativity is obvious (max is commutative). Associativity is also obvious (max is associative). Idempotence holds because `max(x, x) = x`.

4.2.2 PN-Counter (Positive-Negative Counter)

A counter that supports both increment and decrement.

The state is a pair of G-Counters: one tracks increments (`P`), the other tracks decrements (`N`).

- **Query**: `P.query() - N.query()`.
- **Update**: increment by n goes to `P[id] += n`; decrement by n goes to `N[id] += n`. Note that the entries in `N` still only grow; they just represent accumulated decrements.
- **Merge**: merge `P` and `N` independently using the G-Counter merge.

The PN-Counter is built entirely from two G-Counters. The CRDT properties follow from the properties of G-Counter (merging each component preserves commutativity, associativity, and idempotence).

4.2.3 LWW-Register (Last-Writer-Wins Register)

A single-valued register that can be overwritten. Conflicting writes are resolved by keeping the one with the highest timestamp.

The state is a pair `(value, timestamp)`, where `timestamp` is an integer chosen at write time.

- **Query**: return `value`.
- **Update (set value v)**: generate a new timestamp (see below), and if it is greater than the current `timestamp`, replace both `value` and `timestamp`.
- **Merge**: compare timestamps. The merged state has the `(value, timestamp)` pair with the higher timestamp.

The subtlety is the timestamp. You cannot use wall-clock time, because clocks on different machines

are not perfectly synchronized — two writes can happen in a "wrong" order according to wall clocks. The standard trick is to use a *hybrid timestamp*: an integer pair $(t, \texttt{node_id})$, ordered lexicographically. t is the wall-clock time in milliseconds, and `node_id` is the server's id. The `node_id` breaks ties when two servers happen to write at exactly the same millisecond, giving a total order on timestamps that is deterministic even if clocks are out of sync. Two writes that happened "at the same time" on two different servers are resolved by picking one of them consistently; the choice is arbitrary, but both servers will make the same choice because both see the same timestamps.

Commutativity, associativity, and idempotence all follow from the fact that the merge is "keep the one with the higher timestamp," which is a deterministic max.

4.2.4 OR-Set (Observed-Remove Set)

A set that supports both `add` and `remove`. The tricky property it guarantees is: if element e is added after it was last removed, then after convergence, e is in the set. If e was removed after it was last added, then e is not in the set. And concurrent add/remove of the same element is resolved in favor of the add.

The state is structured as two internal sets: `added` and `removed`. Both are sets of pairs of the form `(element, unique_tag)`, where `unique_tag` is a globally unique identifier generated at each add (e.g., a UUID, or a $(\texttt{node_id}, \texttt{counter})$ pair).

- **Query**: return the set of elements e such that at least one `(e, tag)` is in `added` and no `(e, tag)` with the same `tag` is in `removed`. In other words: an element is "in the set" if at least one of its add-tags has not been cancelled by a remove-tag.
- **Add element** e **(on node** `id`**)**: generate a new unique tag, and insert `(e, tag)` into `added`.
- **Remove element** e: for every `(e, tag)` currently in `added`, insert `(e, tag)` into `removed`. (Note: only the tags the removing server can see. Tags it does not yet know about — because they were added on another server whose gossip has not yet arrived — are not cancelled, which is exactly the add-wins property.)
- **Merge**: take the union of both `added` sets, and the union of both `removed` sets.

The mental model: every `add` creates a new, globally unique "observation" of the element. A `remove` is really "cancel all the observations I currently know about." If a concurrent `add` happens on another node, it creates a new observation that the remover did not know about, and that new observation survives the merge — the element ends up back in the set, because one of its tags is in `added` and not in `removed`.

Commutativity, associativity, and idempotence all follow from the fact that set union has those properties.

The OR-Set is the most subtle of the four CRDTs. Spend extra time writing tests for it, especially tests that exercise concurrent add-then-remove and remove-then-add patterns.

4.3 Why Not Vector Clocks

If you read about CRDTs online, you will encounter *vector clocks*, a mechanism for tracking causal relationships between events. Some CRDTs use them. For this project, you do not need them: G-Counter tracks contributions per node (essentially a specialized vector clock for counts), LWW-Register uses hybrid timestamps, and OR-Set uses unique tags. Vector clocks would be a generalization, but they are out of scope. If you implement them as a bonus, note that they are a useful primitive for causality tracking but are not themselves a CRDT.

5 The Gossip Protocol

CRDTs tell us *what* it is safe to merge. The gossip protocol tells us *how* and *when* to exchange states between servers. You will implement a simple anti-entropy gossip protocol: every server, periodically, picks another server at random and sends its entire state; the receiver merges the incoming state into its local state.

5.1 The Loop

On each server, a background thread does the following:

1. Sleep for 1,000 milliseconds (the gossip interval).
2. Pick one of the other servers at random. If this server is the only one up (all others unreachable), skip this round.
3. Serialize the entire local state — every CRDT for every key — into a binary message.
4. Open a TCP connection to the chosen peer's gossip port.
5. Send the serialized state.
6. Receive the peer's serialized state in return.
7. Merge the received state into the local state, key by key. For each key that appears only on the receiver, keep it. For each key that appears only on the sender, add it. For each key that appears on both, merge the two CRDT states using the appropriate merge function.
8. Close the connection.

That is the entire protocol. There is no leader, no term, no handshake, no heartbeat. Every exchange is standalone and symmetric: both parties end up with the same merged state.

5.2 Why This Converges

With n servers and a gossip interval of T, a single update injected at one server reaches all other servers in $O(\log n)$ rounds with high probability (because each round roughly doubles the number of servers that know about the update). In a three-node cluster this means typically one to two gossip cycles — one to three seconds of wall time.

The convergence proof relies on the CRDT merge properties: because the merge is commutative, associative, and idempotent, it does not matter in what order or how many times states are exchanged. Eventually every pair of servers will have gossiped at least once with every other server, and at that point every server has every update.

5.3 Network Failures

A gossip round can fail for many reasons: the chosen peer is down, the network drops the connection, the peer crashes mid-exchange, and so on. The correct response to any of these is the same: log the failure, pick a different peer on the next gossip cycle. No retries, no state tracking, no recovery. The beauty of gossip is that it is self-healing: as long as every server eventually gossips successfully with every other server, convergence is guaranteed.

A particularly interesting case is a network partition that isolates one or more servers from the rest. During the partition, the isolated servers continue to accept writes (because there is no leader election to fail). The two sides of the partition accumulate independent state. When the partition heals and gossip resumes, the first exchange between the two sides transfers all the accumulated state, and the merge function automatically combines them. No special code handles this case; it is simply a consequence of the protocol.

6 The Key-Value Store

On top of the four CRDTs and the gossip protocol, you build a small key-value store where each key is associated with a CRDT of a declared type.

6.1 Commands

Clients talk to any server on its client port. The protocol is plain text, one command per line, same style as the other Project 02 manuals.

- `CREATE key type` — Create a new key with the given CRDT type. `type` is one of `GCOUNTER`, `PNCOUNTER`, `LWWREG`, or `ORSET`. Fails if the key already exists on this server.
- `INC key n` — Increment a G-Counter or PN-Counter by n. Fails on LWW-Register or OR-Set. For a PN-Counter, n may be negative (meaning decrement).
- `SET key value` — Set the value of an LWW-Register. Fails on any other type.
- `ADD key elem` — Add an element to an OR-Set. Fails on any other type.
- `REMOVE key elem` — Remove an element from an OR-Set. Fails on any other type.
- `GET key` — Query the current value of a key. For G-Counter, returns the sum. For PN-Counter, returns the signed sum. For LWW-Register, returns the string value. For OR-Set, returns the current set membership, one element per line.

- `\list` — List all keys on this server with their types and current values.
- `\peers` — List known peers and the last successful gossip time with each.
- `\sync` — Force an immediate gossip round with a randomly chosen peer. Useful for testing without waiting for the gossip timer.
- `QUIT`.

6.2 Handling New Keys from Gossip

A key may be created on one server and then arrive at another server via gossip before the second server has ever heard of it. When this happens, the receiving server simply adds the key to its own keyspace with the CRDT type indicated in the gossip message. There is no need to synchronize the creation operation separately; gossip handles it automatically.

If two servers create a key with the same name but different types simultaneously, the result is an error at merge time: you cannot merge a G-Counter with an OR-Set. This should never happen in correct usage, but your server must detect it and report a clear error rather than crashing. A simple approach is to include the CRDT type in the serialized form of each key, and to reject a merge when the types do not match.

7 The Wire Format

The client-facing protocol is plain text (see Section 6.1). The gossip protocol is a binary format, because gossip messages can be large (every key on every node). All integers are little-endian. All strings are stored as a 4-byte length prefix followed by that many bytes.

7.1 Gossip Message

```
Offset  Size   Field           Description
------  -----  --------------  --------------------------------
  0      4     magic           ASCII bytes 'G','S','S','P'
  4      4     version         uint32, must be 1
  8      4     sender_id_len   uint32
 12      ...   sender_id       bytes
 ...     4     key_count       uint32
 ...     ...   key records     repeated key_count times
```

Each key record is:

```
  0      4     key_len         uint32
  4      ...   key bytes
 ...     1     crdt_type       uint8 (1=GCOUNTER, 2=PNCOUNTER,
                                      3=LWWREG, 4=ORSET)
 ...     4     state_len       uint32
 ...     ...   serialized CRDT state (format depends on type)
```

7.2 CRDT Serialization

Each CRDT type has its own serialization format:

- **G-Counter**: 4-byte entry count, followed by entries. Each entry is a 4-byte node id length, the node id bytes, and an 8-byte signed integer.
- **PN-Counter**: the G-Counter serialization for `P`, followed by the G-Counter serialization for `N`.
- **LWW-Register**: an 8-byte integer timestamp, a 4-byte node id length, the node id bytes, a 4-byte value length, and the value bytes.
- **OR-Set**: a 4-byte count of `added` entries, then the entries (each: 4-byte element length, element bytes, 4-byte tag length, tag bytes); then a 4-byte count of `removed` entries, then the entries in the same format.

The entire gossip message has a trailing 8-byte CRC64 over everything before it. On receipt, verify the CRC; if it does not match, drop the message and do not merge anything.

8 Phase 1 — The Four CRDTs in Isolation

The goal of Phase 1 is to implement the four CRDT types as pure in-memory data structures, with no network and no key-value store. Everything is tested in isolation with unit tests that verify the three merge properties.

Build these components:

- A `GCounter` type with `new`, `increment`, `query`, `merge`, `serialize`, and `deserialize`.
- A `PNCounter` type implemented on top of two `GCounter` instances.
- An `LWWRegister` type with hybrid timestamps. The timestamp is (`wall_clock_ms`, `node_id_string`), compared first by the millisecond, then lexicographically on the id.
- An `ORSet` type. Unique tags can be generated as a pair of (node id, counter), where the counter increases monotonically per node.
- Unit tests that, for each type, verify commutativity, associativity, and idempotence over 100 random state pairs or triples. Generate the states by applying random updates (with reasonable distributions) to freshly created CRDTs.
- Serialization and deserialization round-trip tests: for each type, serialize a state, deserialize it, and verify that every subsequent operation on the deserialized state behaves identically to the original.

At the end of Phase 1 you have a library of working CRDTs that you have proven (by testing) are correctly implemented. This is essential groundwork: if any CRDT is broken, the cluster in Phase 3 will not converge, and debugging it through the gossip protocol is significantly harder than debugging it in unit tests.

9 Phase 2 — The Key-Value Store with Client Protocol

The goal of Phase 2 is a single-node key-value store that uses the four CRDT types and exposes them over the client protocol. No gossip, no network between servers, just one server accepting client commands and applying them to CRDTs.

Build these components:

- A keyspace: a hash map from key name to `(type, crdt_state)`.
- A TCP server that accepts concurrent clients. One thread per client is sufficient.
- A parser for the client protocol (Section 6.1). It is a simple line-based format.
- Handlers for all the commands: `CREATE`, `INC`, `SET`, `ADD`, `REMOVE`, `GET`, `\list`, `\peers` (reports "no peers" for now), `\sync` (no-op for now), `QUIT`.
- A mutex on the keyspace (the single-thread discipline of Redis is not needed here because the gossip thread will need to touch the same data as the client threads).
- A command-line client `crdtkv-cli`.

At the end of Phase 2, a single server should support all four CRDT types from a single client's point of view. You cannot yet demonstrate convergence (only one server exists), but the full client-facing API is working.

10 Phase 3 — Gossip and Convergence

The goal of Phase 3 is to connect the servers via gossip, demonstrate convergence across a three-node cluster, and produce a benchmark that quantifies it.

Build these components:

- The gossip port: a second TCP listener on a separate port from the client port.
- A background thread that runs the gossip loop (Section 5.1) every 1,000 ms, picking a random peer and exchanging states.
- The gossip message format (Section 7) including the CRC64.
- The `\sync` command that triggers an immediate gossip exchange.
- Merge handlers: given a deserialized key from the wire, look up the local state for that key, and merge or insert as appropriate. Reject type mismatches with an error.
- Update to `\peers` to show the last successful gossip time per peer.

- Unit and integration tests for the gossip path: create two servers in the same process (on different ports), let them gossip, verify that a write on one becomes visible on the other after one gossip cycle.

10.1 The Convergence Test

Write a test driver (shell script or separate binary) that exercises a real three-node cluster. The test runs the following scenarios and verifies the expected outcome after each:

1. Start three nodes. Create a G-Counter on node A. Increment it 50 times across all three nodes (roughly evenly distributed). Wait 5 seconds. Verify that all three nodes report the same total (which must equal the sum of all 50 increments, exactly).
2. Start three nodes. Create an OR-Set on node A. Concurrently, add 10 distinct elements on node A and 10 distinct elements on node B while node C does nothing. Then remove one element (say, the first one added on A) on node C. Wait 5 seconds. Verify that all three nodes have the same set, that it contains 19 elements (the 20 adds minus the 1 remove), and that the specific removed element is gone.
3. Start three nodes. Create an LWW-Register on node A and set it to "apple." Immediately set it to "banana" on node B and "cherry" on node C (all three writes happen within a few milliseconds). Wait 5 seconds. Verify that all three nodes converge to the same value — the exact value is not specified (it depends on timestamp tiebreaking), but all three nodes must agree.
4. **Partition test.** Start three nodes. Create a PN-Counter and increment it a few times from each server to establish a known state. Now simulate a partition that isolates node C from nodes A and B (you can implement this in the server itself with a command-line flag or a special command, e.g., `\partition C`, that causes the server to reject gossip connections from or to C). Increment the counter independently on both sides. Wait 5 seconds. Verify that A and B converge between themselves but disagree with C. Heal the partition. Wait 5 more seconds. Verify that all three nodes converge, and that the final count is the sum of the individual contributions from each node, with nothing lost and nothing double-counted.
5. **Crash test.** Start three nodes. Write several keys. Kill node B abruptly. Continue writing on A and C. Restart B with a fresh, empty state. Verify that B re-acquires the keys via gossip within a couple of cycles and that all three nodes converge to the same state. (Note: this test assumes your server does not persist anything to disk. If you implement persistence as a bonus, modify the test accordingly.)

The convergence test driver is the most important deliverable of Phase 3. A submission that appears to work in the happy path but fails any of these scenarios will receive a reduced grade proportional to the number of failures.

10.2 Benchmark

Alongside the correctness tests, run a throughput benchmark:

- Start three nodes. Create a G-Counter.
- Fire 10,000 `INC` commands across the cluster (distributed roughly evenly among the three nodes). Measure wall-clock time. Report the aggregate increments per second.
- Measure the time between the last increment and the first moment when all three nodes agree on the final value ("convergence time").
- Repeat for the other three CRDT types with analogous workloads.

The expected result is that throughput is essentially limited only by the per-node command processing speed (several tens of thousands of operations per second), because there is no coordination — writes return immediately. Convergence time should be on the order of 1 to 3 gossip intervals (1 to 3 seconds at a 1-second gossip interval).

11 Deliverables and Submission

11.1 What to Submit

A single zip archive containing:

- The complete source tree of your implementation, including the server (`crdtkv`), the client (`crdtkv-cli`),

and the convergence test driver.

- A `Makefile` (or `CMakeLists.txt`) that builds all binaries with a single command on a recent Ubuntu LTS.
- A `README.md` with build instructions, the list of supported commands, instructions for starting a three-node cluster in each of the deployment options in Section 3, and any known limitations.
- A `design.pdf` document of at most 12 pages describing: the four CRDT types with their state representation and merge functions in your implementation, the proof sketches (or at least convincing arguments) for commutativity, associativity, and idempotence of each merge, the gossip protocol, the wire format, the convergence test scenarios and the observed behavior, and the benchmark results with throughput and convergence-time numbers.
- A `tests/` directory with unit tests for each CRDT type (including the three merge properties), the serialization round-trip, and the convergence test driver.
- A `docker-compose.yml` or equivalent for whichever deployment option you used.

11.2 Archive Naming

`Group[Number]_Project02_CRDT.zip`, for example `Group17_Project02_CRDT.zip`.

11.3 Demonstration

Each group will present a live demonstration during the week following the submission deadline. The demonstration consists of three parts. First, a brief walkthrough of the four CRDT types using the design document, with particular focus on explaining why each merge function has the three required properties. Second, a live execution of the convergence test on a running three-node cluster, with the instructor free to propose additional concurrent-write scenarios on the spot ("simultaneously increment the counter on all three nodes, predict the result, check"). Third, a short oral examination in which each group member will be asked to explain, without notes, why a specific CRDT converges, typically this takes the form of "walk me through a scenario on paper where two nodes do conflicting operations and explain the final state after merge." All group members must be present. Missing the demonstration results in a zero for Phase 3.

12 Bonus Opportunities

Each of the following is independent. Bonus credit is awarded on top of the base grade up to a maximum of 15 percentage points. To claim bonus credit, your design document must include a dedicated section explaining which items you attempted and how you verified them.

12.1 Persistence via State Snapshots (up to 5%)

The base project loses all state when a server is killed. Add periodic snapshots: every N seconds, serialize the entire keyspace to a file, and on startup, load the most recent snapshot if one exists. The snapshot format can reuse the CRDT serialization from the gossip protocol. Your convergence test scenario 5 should be updated to run with persistence enabled and verify that a restarted node starts from its pre-crash state.

12.2 Delta-State Gossip (up to 5%)

The base project sends the entire state on every gossip round, which is wasteful. Implement delta-state gossip: each server remembers which updates it has sent to each peer (by tracking a per-peer "high-water mark" in some form), and on each gossip round only sends the updates the peer has not yet seen. The exact mechanism for tracking this depends on which CRDTs you are gossiping — describe yours in the design document. Your benchmark must include a comparison of bytes-per-gossip-round between full state and delta state.

12.3 A Fifth CRDT: 2P-Set or MV-Register (up to 5%)

Implement one additional CRDT beyond the four required. Options include: the **2P-Set** (two-phase set), which supports add and remove but disallows re-adding a removed element (simpler than OR-Set, and a good contrast to study); or the **Multi-Value Register**, which preserves all concurrent writes rather than picking a winner (useful for applications that want to surface conflicts to the user, such as shopping carts in Amazon's Dynamo paper). Include full unit tests with the three merge properties, and add it to the client protocol and the gossip wire format.

13 Constraints and Prohibitions

The Following Will Result in Loss of Credit

- Use of any existing CRDT library (Automerge, Yjs, riak_dt, akka-distributed-data, delta-crdts, Redis's CRDB features, etc.). Implement the CRDT types yourself.
- Use of any existing gossip framework or service discovery library (SWIM, memberlist, serf, etc.). The gossip loop in Section 5 is short and must be written from scratch.
- Use of any existing RPC or serialization framework (gRPC, Thrift, Protobuf, Cap'n Proto, FlatBuffers). The gossip wire format is specified in Section 7 and must be implemented with direct byte reads and writes.
- Coordinating writes between servers before acknowledging clients. The whole point of the project is that writes are local and immediate. If your server sends any message to another server before responding to a client's `INC`, `SET`, `ADD`, or `REMOVE`, you have missed the point and Phase 2 will be scored accordingly.
- Skipping the three merge-property tests for any CRDT type. These tests are the only practical check that your implementation is correct, and the design document must refer to them by name when arguing correctness.
- Using wall-clock time directly as a tiebreaker without a node-id component in the LWW-Register. Clock skew on different machines will cause convergence bugs that are very hard to detect.
- Use of Python at any layer, including test harnesses and convergence test drivers. Tests and scripts must be written in C, C++, shell, or JavaScript.
- Submitting code whose commit history shows large, infrequent commits. Your git history should reflect the incremental development of a system of this size.

14 Required Reading

The first two items are required reading before you begin Phase 1. The remaining items are reference material.

1. Shapiro, M., Preguiça, N., Baquero, C., and Zawirski, M. "A Comprehensive Study of Convergent and Commutative Replicated Data Types." INRIA Research Report 7506, 2011. This is the canonical reference for CRDTs. Sections 3 (state-based CRDTs) and 4 (specifications of the basic types: G-Counter, PN-Counter, LWW-Register, OR-Set) cover almost everything in this manual at greater formal depth. Read Sections 1 through 4 before Phase 1. The paper is long and partially formal; skim the formal parts on the first pass and focus on the specifications.
2. Kleppmann, M. *Designing Data-Intensive Applications*, Chapter 5 ("Replication"), specifically the section titled "Multi-Leader Replication" and its subsection on "Automatic Conflict Resolution." A clear, high-level discussion of the scenarios where CRDTs are useful and how they fit into the broader landscape of replication approaches. Read it before Phase 3.
3. Shapiro, M. et al. "Conflict-free Replicated Data Types." *Proceedings of the International Symposium on Stabilization, Safety, and Security of Distributed Systems*, 2011. The shorter conference version of the technical report. Useful if you find the longer version intimidating.
4. DeCandia, G. et al. "Dynamo: Amazon's Highly Available Key-Value Store." *ACM Symposium on Operating Systems Principles*, 2007. The influential paper that introduced the idea of applying eventual consistency to a real production database. Section 4.4 on reconciliation describes the shopping-cart use case that motivated the Multi-Value Register (relevant to one of the bonus options).
5. Preguiça, N., Zawirski, M., Bieniusa, A., Duarte, S., Balegas, V., Baquero, C., and Shapiro, M. "An Optimized Conflict-Free Replicated Set." arXiv:1210.3368, 2012. Optional. A more efficient implementation of OR-Set that uses version vectors instead of unique tags. Useful if you want to understand how production CRDT libraries actually represent OR-Sets.

Penalty Structure

- Missing documentation: –10%
- Security vulnerabilities: –5% per issue
- Poor code quality: –5% to –15%
- Academic integrity violations: Zero grade

Contact & Support

Teaching Assistants:
Jalal Ahmed: `bscs23134@itu.edu.pk`
Khadijah Farooqi: `bscs23128@itu.edu.pk`

Office Hours: Monday & Thursday,
11:30 AM – 2:30 PM